

UNIFECAP
DOCUMENTAÇÃO TÉCNICA COMPLETA

CURSO DE ENGENHARIA DE SOFTWARE

**SISTEMA DE CONTROLE INTELIGENTE DE AR-CONDICIONADO:
DOCUMENTAÇÃO TÉCNICA COMPLETA**

SMART BUILDING
SISTEMA DE CONTROLE INTELIGENTE DE AR-CONDICIONADO:
DOCUMENTAÇÃO TÉCNICA COMPLETA

DOCUMENTAÇÃO TÉCNICA COMPLETA

RESUMO

Este documento apresenta a especificação técnica completa do sistema Smart Building para monitoramento e controle inteligente de sistemas de climatização predial. A solução integra sensores IoT sob protocolo MQTT, backend em FastAPI com arquitetura orientada a eventos, frontend React com visualização em tempo real e módulo de inteligência artificial baseado em Scikit-learn para predição de consumo energético. O projeto abrange todos os aspectos esperados de um sistema profissional: requisitos funcionais e não funcionais, regras de negócio, arquitetura detalhada em múltiplos níveis (C4), modelagem de dados completa com políticas de retenção, especificação de API RESTful documentada, topologia de rede IoT com recomendações de hardware, estratégia de deployment em nuvem com CI/CD, plano abrangente de testes, monitoramento, segurança da informação e plano de contingência. O objetivo primário é a redução de até 30% no consumo energético predial, aliada à automação inteligente e ao conforto dos ocupantes.

Palavras-chave: Internet das Coisas; Smart Building; MQTT

ABSTRACT

This document presents the complete technical specification of the Smart Building system for intelligent monitoring and control of building air conditioning. The solution integrates IoT sensors via MQTT protocol, a FastAPI backend with event-driven architecture, a React frontend with real-time visualization, and an artificial intelligence module based on Scikit-learn for energy consumption prediction. The project covers all aspects expected of a professional system: functional and non-functional requirements, business rules, multi-level detailed architecture (C4), complete data modeling with retention policies, documented RESTful API specification, IoT network topology with hardware recommendations, cloud deployment strategy with CI/CD, comprehensive test plan, monitoring, information security, and contingency plan. The primary objective is a reduction of up to 30% in building energy consumption, combined with intelligent automation and occupant comfort.

Keywords: Internet of Things; Smart Building; MQTT

LISTA DE ILUSTRAÇÕES

Figura 1 – Árvore de diretórios do backend com módulos e arquivos FastAPI	17
Figura 2 – Diagrama C4 – Nível de Contêineres	18
Figura 3 – Diagrama de Sequência – Coleta de Dados IoT	20
Figura 4 – Trecho de tópicos MQTT usados para telemetria	20
Figura 5 – Exemplo de comando via API e MQTT	20
Figura 6 – Diagrama de Sequência – Controle manual	20
Figura 7 – Diagrama de Sequência – Fluxo de Alertas	21
Figura 8 – Diagrama de Sequência – Predição com IA	21
Figura 9 – Árvore de diretórios do backend com módulos e arquivos FastAPI	27
Figura 10 – Diagrama Entidade-Relacionamento (DER)	33
Figura 11 – Diagrama de Sequência – Inicialização	36
Figura 12 – Diagrama de Sequência – Treinamento e Predição	38
Figura 13 – Diagrama de Sequência – Retorno de Anomalia (-1) ou Normal (1)	39

LISTA DE TABELAS

Tabela 1 – Stack tecnológico selecionado para o sistema	13
Tabela 2 – Requisitos funcionais identificados junto aos stakeholders	14
Tabela 3 – Requisitos Não Funcionais	14
Tabela 4 – Definição da estrutura da entidade de usuários do sistema	23
Tabela 5 – Definição da entidade de salas no modelo de dados	23
Tabela 6 – Definição da entidade de dispositivos e sensores do modelo de dados	24
Tabela 7 – Estrutura da entidade de dados de sensores e leituras	24
Tabela 8 – Estrutura da tabela de comandos de dispositivos e sensores	25
Tabela 9 – Estrutura de campos da tabela de alertas de sensores	25
Tabela 10 – Estrutura da tabela de dados de sensores e previsões de consumo	26
Tabela 11 – Estrutura e descrição dos campos da tabela de auditoria de eventos	26
Tabela 12 – Relacionamentos e cardinalidades entre as entidades do sistema	27
Tabela 13 – Endpoints da API para gerenciamento e consulta de sensores	30
Tabela 14 – Endpoints da API para gerenciamento e controle de dispositivos	30
Tabela 15 – Endpoints da API para gerenciamento de alertas	30
Tabela 16 – Endpoints da API para consumo e previsões de energia	31
Tabela 17 – Configurações de parâmetros do banco de dados PostgreSQL	36
Tabela 18 – Métricas de avaliação e metas de desempenho do modelo de regressão	38
Tabela 19 – Protocolos de rede utilizados em cada camada do modelo TCP/IP	43
Tabela 20 – Casos de teste principais baseados nos requisitos funcionais	44
Tabela 21 – Casos de teste principais baseados nos requisitos funcionais	50
Tabela 22 – Métricas de monitoramento e disponibilidade do sistema	51
Tabela 23 – Mapeamento de cenários de falha, probabilidade, impacto e resposta	53
Tabela 24 – Definição de permissões por papel no modelo RBAC	55

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
AC	Air Conditioning (Ar-Condicionado)
AES	Advanced Encryption Standard
API	Application Programming Interface
ARIMA	Auto-Regressive Integrated Moving Average
C4	Modelo C4 de arquitetura de software
CBECS	Commercial Buildings Energy Consumption Survey
CIP	Dados Internacionais de Catalogação na Publicação
DER	Diagrama Entidade-Relacionamento
EMQX	MQTT Broker
ESP32	Microcontrolador com Wi-Fi e Bluetooth integrado
GB	Gigabytes
HTTPS	Hypertext Transfer Protocol Secure
IA	Inteligência Artificial
IOT	Internet of Things (Internet das Coisas)
IP	Internet Protocol
IR	Infravermelho (Infrared)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LGPD	Lei Geral de Proteção de Dados Pessoais
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MQTT	Message Queuing Telemetry Transport
NBR	Norma Brasileira
RAM	Random Access Memory
REST	Representational State Transfer
RF	Requisito Funcional
RMSE	Root Mean Square Error
RN	Regra de Negócio

SUMÁRIO

1 VISÃO GERAL	12
1.1 Descrição	12
1.2 Objetivos	12
1.3 Público-Alvo	13
1.4 Stack Tecnológico	13
2 REQUISITOS	14
2.1 Requisitos Funcionais (RF)	14
2.1.1 Quadro 1 – Requisitos Funcionais	14
2.2 Requisitos Não Funcionais (RNF)	14
2.3 Regras de Negócio (RN)	15
3 ARQUITETURA DO SISTEMA	17
3.1 Padrão Arquitetural	17
3.2 Diagrama C4 – Contexto (Nível 1)	17
3.2.1 Figura 1 – Diagrama C4 – Nível de Contexto	17
3.2.2 @startuml	17
3.3 Diagrama C4 – Contêineres (Nível 2)	18
3.3.1 Figura 2 – Diagrama C4 – Nível de Contêineres	18
3.4 Responsabilidades dos Componentes	18
4 FLUXOS DO SISTEMA	20
4.1 Fluxo de Coleta de Dados IoT	20
4.1.1 Figura 3 – Diagrama de Sequência – Coleta de Dados IoT	20
4.2 Fluxo de Controle Manual	20
4.3 Fluxo de Alertas	21
4.3.1 Figura 5 – Diagrama de Sequência – Fluxo de Alertas	21
4.4 Fluxo de Predição com IA	21
4.4.1 Figura 6 – Diagrama de Sequência – Predição com IA	21
5 MODELAGEM DE DADOS	23
5.1 Entidades Principais	23
5.1.1 Users (Usuários)	23
5.1.2 Rooms (Salas)	23
6 DEVICES (DISPOSITIVOS)	24
6.1 SensorData (Leituras dos Sensores)	24
6.1.1 Commands (Comandos)	24
6.2 Relacionamentos	26
6.2.1 Figura 7 – Diagrama Entidade-Relacionamento (DER)	26

6.3 Ciclo de Vida e Retenção de Dados (Data Lifecycle)	27
6.4 Políticas de Segurança dos Dados	28
7 ESPECIFICAÇÃO DA API RESTFUL	29
7.1 Autenticação e Autorização	29
7.1.1 POST /api/v1/auth/login	29
8 ENDPOINTS PRINCIPAIS	30
8.1 Sensores	30
8.2 Dispositivos (AC)	30
8.3 Alertas	30
8.4 Consumo e Predição	30
9 DOCUMENTAÇÃO SWAGGER	32
10 COMPONENTES TÉCNICOS	33
10.1 Backend (FastAPI)	33
10.1.1 Principais Dependências (Requirements.txt)	34
10.2 Frontend (React)	34
10.2.1 Estrutura de Componentes Principais	34
10.2.2 Principais Bibliotecas (Package.json)	34
11 MQTT – PADRÃO DE NOMENCLATURA DE TÓPICOS	35
11.1 Telemetria (Sensores)	35
12 COMANDOS (ATUADORES):	36
12.1 Status (Feedback):	36
12.2 Banco de Dados (PostgreSQL)	36
12.3 Configurações Recomendadas Para Ambiente de Produção (Supabase / VPS com 4 GB RAM):	36
12.3.1 Índices Adicionais Para Performance:	36
13 INTELIGÊNCIA ARTIFICIAL	38
13.1 Modelo de Predição de Consumo	38
13.1.1 Dados de Entrada (Features):	38
13.1.2 Métricas de Avaliação e Metas:	38
13.1.3 Política de Retreinamento:	38
13.2 Detecção de Anomalias	39
14 INICIALIZAÇÃO	40
15 TREINAMENTO E PREDIÇÃO	41
16 RETORNO: -1 = ANOMALIA (ALERTA GERADO), 1 = NORMAL	42
17 TOPOLOGIA DE REDE IOT	43
17.1 Protocolos de Comunicação	43

17.1.1 Quadro 4 – Protocolos de Comunicação Utilizados	43
17.2 Distribuição de Dispositivos	43
17.3 Camadas de Rede	43
17.4 Recomendações de Hardware	44
17.4.1 Quadro 5 – Especificações de Hardware Recomendadas (100 dispositivos IoT)	44
17.5 Especificação dos Módulos Sensores e Atuadores (Edge)	44
17.6 Lógica de Resiliência (Offline Fallback)	44
18 DEPLOYMENT E INFRAESTRUTURA	46
18.1 Plataformas de Hospedagem	46
18.1.1 Frontend (React) – Vercel	46
18.1.2 Backend (FastAPI) – Render	46
18.1.3 Banco de Dados (PostgreSQL) – Supabase	46
18.1.4 MQTT Broker – EMQX Cloud	46
18.2 Docker Compose (Ambiente Local e Desenvolvimento)	46
19 PIPELINE DE CI/CD	48
20 ESTRATÉGIA DE TESTES	49
20.1 Testes Unitários	49
20.1.1 Frontend (Jest + React Testing Library)	49
20.2 Testes de Integração	49
20.3 Testes de Carga e Performance	49
20.3.1 Cenários	49
20.3.2 Critérios de Aceitação	49
20.4 Testes de Aceitação (UAT)	50
20.4.1 Quadro 7 – Casos de Teste – Aceitação	50
21 MONITORAMENTO E OBSERVABILIDADE	51
21.1 Métricas do Sistema	51
21.1.1 Quadro 8 – Métricas de Monitoramento	51
21.2 Logging	51
21.3 Alertas Operacionais	51
22 PLANO DE CONTINGÊNCIA E DISASTER RECOVERY	53
22.1 Análise de Riscos	53
22.1.1 Quadro 9 – Plano de Contingência – Cenários e Respostas	53
22.2 Procedimentos de Recuperação	53
22.2.1 Queda do Broker MQTT	53
22.2.2 Queda do Banco de Dados	54
22.2.3 Perda Total do Ambiente Cloud (Desastre)	54

22.2.4 Backups	54
23 SEGURANÇA DA INFORMAÇÃO	55
23.1 Criptografia e Comunicação	55
23.2 Controle de Acesso	55
23.3 Conformidade com a LGPD	55
24 DECISÕES DE PROJETO	57
24.1 Por que MQTT?	57
24.1.1 Alternativas Avaliadas e Motivos de Descarte	57
24.2 Por que FastAPI?	57
24.2.1 Alternativas:	57
24.3 Por que React?	57
24.3.1 Alternativas:	57
24.4 Por que Scikit-learn?	58
24.4.1 Alternativas:	58
REFERÊNCIAS	59

1 VISÃO GERAL

1.1 Descrição

O presente documento descreve, de forma exaustiva, o sistema *Smart Building* voltado ao monitoramento e controle inteligente de sistemas de ar-condicionado em edifícios corporativos e residenciais de múltiplos andares. A solução integra sensores IoT de baixo custo, um *backend* robusto e escalável, painel de controle (*dashboard*) interativo e um módulo de inteligência artificial para otimização do consumo energético.

O controle de climatização representa, segundo a *U.S. Energy Information Administration* (2023), aproximadamente 40% do consumo elétrico total em edificações comerciais. Nesse contexto, sistemas inteligentes que aliam automação preditiva, sensoriamento de presença e aprendizado de máquina podem gerar economias significativas sem comprometer o conforto térmico dos ocupantes (PEDREGOSA et al., 2011).

A arquitetura adota o protocolo MQTT como espinha dorsal da comunicação entre dispositivos de campo e a nuvem, garantindo baixa latência, *throughput* elevado e desacoplamento entre componentes — características essenciais em cenários de Internet das Coisas (MQTT FOUNDATION, 2026).

1.2 Objetivos

O projeto persegue os seguintes objetivos estratégicos e operacionais:

- **Eficiência energética:** reduzir o consumo elétrico dos sistemas de climatização em até 30% por meio de desligamento automático baseado em presença, bloqueio com janelas abertas e ajuste dinâmico de *set-points*;
- **Automação inteligente:** eliminar a dependência de intervenção humana para rotinas de climatização, aplicando as regras de negócio (RN01 a RN10) de forma autônoma;
- **Geração de *insights*:** utilizar modelos de aprendizado de máquina para prever o consumo das próximas 24 horas e recomendar ações de economia;
- **Conforto do usuário:** manter a temperatura ambiente dentro da faixa de 23°C a 25°C durante os horários de ocupação, respeitando a Norma Regulamentadora NR-17;

- **Manutenção preventiva:** detectar anomalias em sensores e equipamentos, emitindo alertas que permitam a atuação da equipe técnica antes da falha completa.

1.3 Público-Alvo

Os principais atores que interagem com o sistema são:

- **Gestores Prediais:** responsáveis pelo monitoramento global do edifício, visualização de relatórios gerenciais e definição de políticas de economia;
- **Equipe Técnica:** encarregada da manutenção corretiva e preventiva, recebendo notificações de anomalias (*troubleshooting*);

Supervisores: recebem alertas configuráveis via *dashboard* e e-mail, bem como relatórios periódicos de desempenho.

1.4 Stack Tecnológico

O *stack* tecnológico foi selecionado com base nos critérios discutidos na seção 15 e é composto por:

Tabela 1 – Stack tecnológico selecionado para o sistema

Camada	Tecnologia	Hospedagem
Frontend	React.js 18	Vercel
Backend	FastAPI (Python 3.11+)	Render / Railway
Banco de Dados	PostgreSQL 15+	Supabase
Mensageria	MQTT (Mosquitto / EMQX)	EMQX Cloud ou VPS própria
IA / ML	Scikit-learn 1.3+	Integrado ao backend
Infraestrutura	Docker + GitHub Actions	Nuvem pública

Fonte: Elaborado pelo autor

2 REQUISITOS

2.1 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as funcionalidades que o sistema deve prover. O Quadro 1 enumera os doze requisitos identificados junto aos *stakeholders*.

2.1.1 Quadro 1 – Requisitos Funcionais

Tabela 2 – Requisitos funcionais identificados junto aos stakeholders

ID	Requisito
RF01	Monitoramento em tempo real de temperatura/umidade
RF02	Dashboard web com visualização de dispositivos
RF03	Controle remoto (ligar/desligar/set-point)
RF04	Deteção de ocupação via sensores
RF05	Sistema de alertas inteligentes
RF06	Notificações via dashboard/e-mail
RF07	Registro histórico de operações
RF08	Relatórios de consumo (diários/semanais/mensais)
RF09	Predição de consumo com IA (24h)
RF10	Recomendações automáticas de economia
RF11	API RESTful para integração com terceiros
RF12	Gestão de usuários e autenticação

Fonte: Elaborado pelo autor (2026).

2.2 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais (RNF) definem atributos de qualidade do sistema. O Quadro 2 apresenta os dez RNF estabelecidos.

Quadro 2 – Requisitos Não Funcionais

Tabela 3 – Requisitos Não Funcionais

ID	Requisito	Meta
RNF01	Latência de atualização	≤ 5 segundos (sensor → dashboard)
RNF02	Escalabilidade	100+ dispositivos simultâneos
RNF03	Autenticação	JWT + OAuth 2.0
RNF04	Segurança de dados	HTTPS + AES-256 (em repouso e trânsito)
RNF05	Disponibilidade	99% de uptime (SLA mensal)
RNF06	Padronização de API	RESTful + Swagger / OpenAPI 3.0
RNF07	Persistência	PostgreSQL com failover automático
RNF08	Performance de consultas	Consultas simples < 2 segundos
RNF09	Usabilidade	Interface responsiva e acessível (WCAG 2.1)
RNF10	Manutenibilidade	Clean Code + arquitetura modular

Fonte: Elaborado pelo autor (2026).

2.3 Regras de Negócio (RN)

As regras de negócio (RN) refletem políticas operacionais do edifício e são aplicadas pelo *backend* de forma automática:

- **RN01:** Desligamento automático do ar-condicionado após 15 minutos sem presença detectada na sala;
- **RN02:** Bloqueio do funcionamento se a janela permanecer aberta por mais de 5 minutos consecutivos;
- **RN03:** Temperatura ideal de operação: *set-point* entre $23\text{text}^{\circ}\text{C}$ e $25\text{text}^{\circ}\text{C}$, ajustável por sala;
- **RN04:** Comandos manuais emitidos pelo gestor no *dashboard* sobrescrevem os comandos automáticos;
- **RN05:** Alerta emitido quando o consumo do dispositivo ultrapassar o limite configurado por sala;
- **RN06:** Respeitar os horários de funcionamento do edifício (ex.: 07h00 às 20h00 em dias úteis);
- **RN07:** Retreinar o modelo de IA diariamente (às 02h00) com os dados mais recentes dos últimos 30 dias;
- **RN08:** Validar todos os dados recebidos (intervalo plausível de temperatura, umidade, *timestamp*) antes de armazená-los;
- **RN09:** Manter trilha de auditoria completa de todas as ações executadas (quem, o quê, quando e *payload*);
- **RN10:** Priorizar a eficiência energética nas decisões automatizadas, respeitando os limites de conforto.

3 ARQUITETURA DO SISTEMA

3.1 Padrão Arquitetural

O sistema adota arquitetura **orientada a eventos** com o padrão **Publicador-Assinante (Pub/Sub)** sobre o protocolo MQTT, complementado por comunicação síncrona REST para o *frontend*. Essa abordagem híbrida garante comunicação assíncrona e desacoplada entre os dispositivos de campo e o núcleo do sistema, enquanto oferece uma API previsível para o *dashboard*.

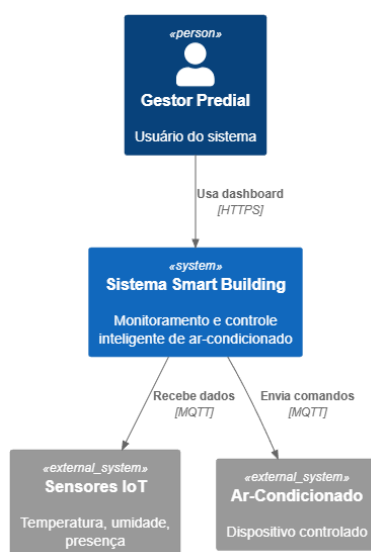
O fluxo de dados segue o seguinte roteiro:

- Sensores IoT publicam leituras em tópicos MQTT predefinidos (telemetria);
- O *backend* (FastAPI) se inscreve nos tópicos de interesse e persiste as leituras no PostgreSQL;
- Comandos de controle emitidos pelo gestor são publicados nos tópicos dos atuadores;
- Atualizações em tempo real são entregues ao *dashboard* React via *WebSocket* (canal mantido pelo Supabase Realtime).

3.2 Diagrama C4 – Contexto (Nível 1)

O diagrama de contexto (Nível 1 do modelo C4) apresenta o sistema como uma "caixa preta" e mostra como ele se relaciona com os atores externos (BROWN, 2026).

3.2.1 Figura 1 – Diagrama C4 – Nível de Contexto



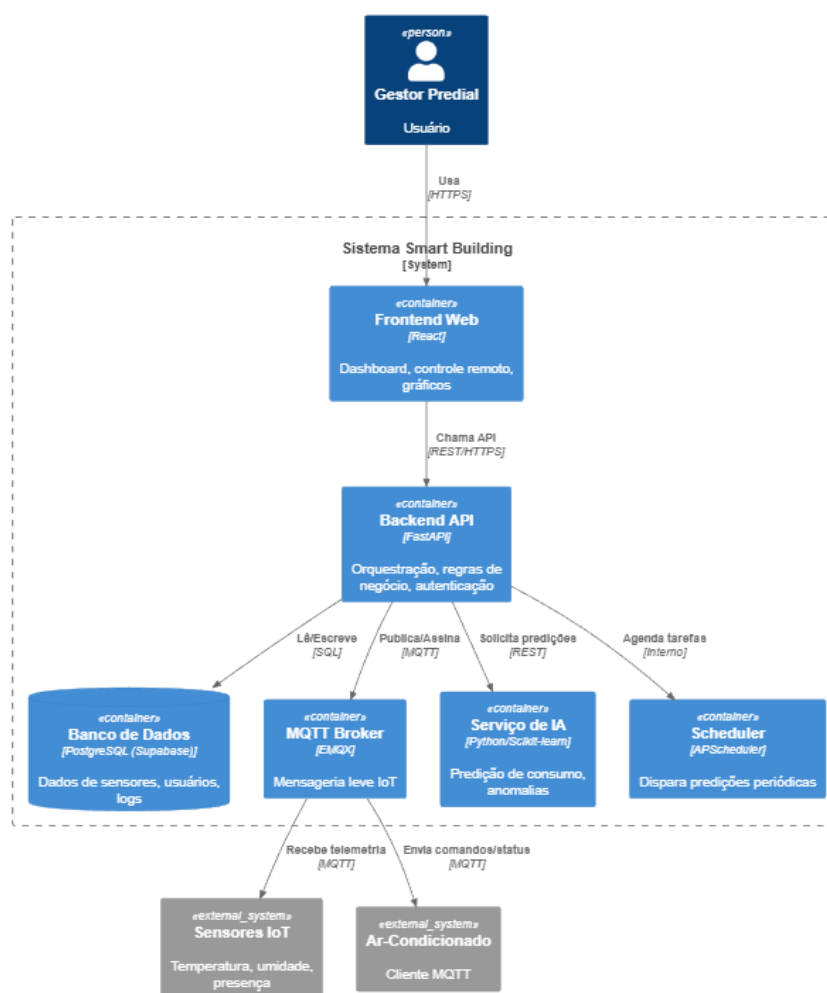
Descrição da figura: O diagrama mostra, no centro, o retângulo azul representando o "Sistema Smart Building". À esquerda, o ator "Gestor Predial"

(boneco azul) interage com o sistema por meio de um *dashboard web*. Na parte inferior, um segundo ator "Equipe Técnica" recebe alertas e acessa relatórios. À direita, o sistema externo "API Meteorológica" fornece dados de temperatura externa. Na parte superior, os "Sensores IoT" e "Ar-Condicionados" comunicam-se com o sistema via protocolo MQTT.

3.3 Diagrama C4 – Contêineres (Nível 2)

O diagrama de contêineres (Nível 2) decompõe o sistema em seus principais artefatos executáveis (contêineres *Docker*, bancos de dados, serviços).

3.3.1 Figura 2 – Diagrama C4 – Nível de Contêineres



Fonte: Elaborado pelo autor (2026).

Descrição da figura: O diagrama exhibe seis contêineres principais dentro do retângulo azul do "Sistema Smart Building". O "Frontend Web (React)" no canto superior esquerdo comunica-se com o "Backend API (FastAPI)" via HTTPS/REST. O

backend, por sua vez, conecta-se ao "Supabase DB (PostgreSQL)", ao "MQTT Broker (EMQX)" e ao "Serviço de IA (Python)". Um componente "Scheduler" executa tarefas periódicas, como retreinamento do modelo. Os "Sensores IoT" externos publicam no *broker*, e os "Ar-Condicionados" subscrevem-se a tópicos de comando.

3.4 Responsabilidades dos Componentes

Cada contêiner possui responsabilidades claramente delimitadas:

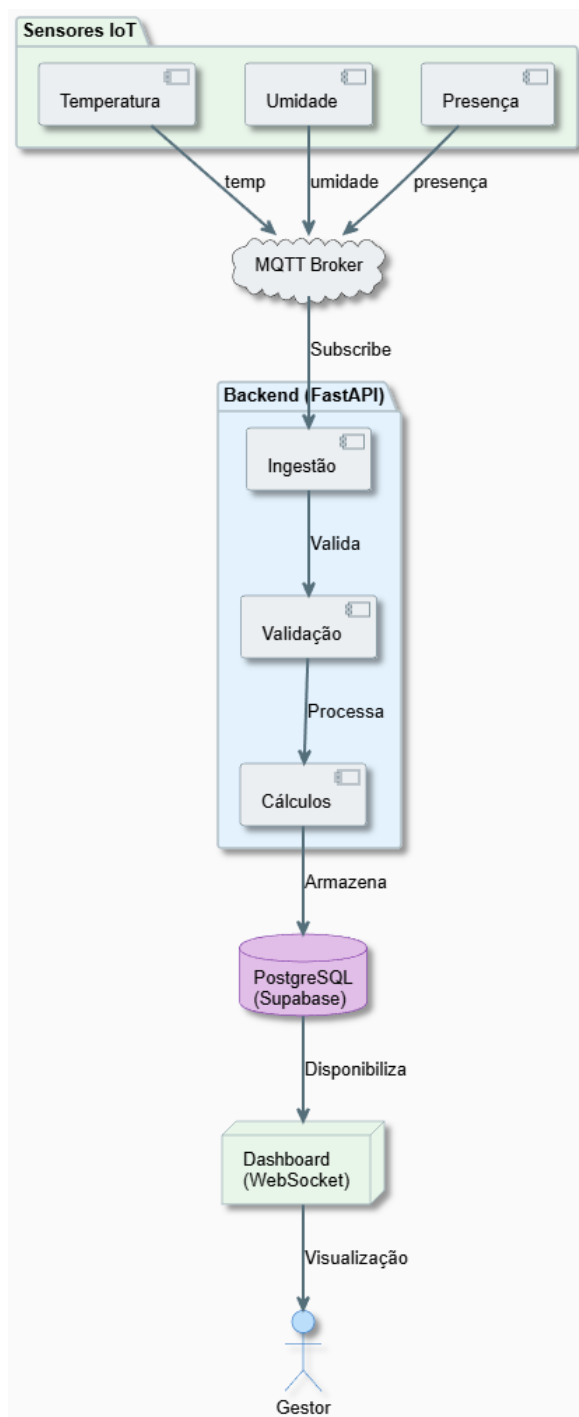
- **Frontend (React):** renderização do *dashboard* interativo; controle remoto dos dispositivos; exibição de alertas em tempo real; visualização de relatórios gráficos (Recharts); autenticação do usuário (login/logout) e armazenamento seguro do *token JWT* em *httpOnly cookie*;
- **Backend (FastAPI):** orquestração de todos os fluxos de negócio; aplicação das regras RN01 a RN10; integração bidirecional com o *broker MQTT* (subscrição de telemetria e publicação de comandos); autenticação e autorização via JWT + OAuth 2.0; exposição da API RESTful documentada; validação de dados com Pydantic; cache opcional com Redis;
- **MQTT Broker (EMQX):** roteamento de mensagens no modelo Pub/Sub; suporte à Qualidade de Serviço (QoS 0, 1 e 2); autenticação de clientes por certificado TLS; *bridge* para outros *brokers* em cenários multiedifício; fila de mensagens *offline* para dispositivos temporariamente desconectados;
- **Banco de Dados (PostgreSQL):** persistência transacional de todas as leituras, comandos, alertas e previsões; armazenamento de configurações por sala e dispositivo; suporte a *triggers* para atualização automática de *status*; particionamento de tabelas por mês para otimização de consultas;
- **Serviço de IA:** treinamento diário do modelo de regressão; previsão do consumo para as próximas 24 horas; detecção de anomalias em tempo real nas leituras dos sensores; geração de recomendações automáticas de economia (ex.: "Sugerimos elevar o *setpoint* da Sala 301 para $24\text{text}^{\circ}\text{C}$, economia estimada de $2,3\text{textkWh}$ ").

4 FLUXOS DO SISTEMA

4.1 Fluxo de Coleta de Dados IoT

O fluxo de coleta de dados de sensores IoT obedece a uma latência máxima de 5 segundos (RNF01) desde a leitura física até a exibição no *dashboard*. A sequência de eventos é detalhada no diagrama a seguir.

4.1.1 Figura 3 – Diagrama de Sequência – Coleta de Dados IoT



Fonte: Elaborado pelo autor (2026).

Descrição da figura: O diagrama mostra a interação entre quatro entidades: "Sensor IoT", "MQTT Broker", "Backend" e "Dashboard". O sensor publica uma mensagem de telemetria no tópico `telemetry/.../temperature`. O *broker* encaminha a mensagem ao *backend*, que está inscrito no tópico. O *backend* valida os dados (intervalo plausível), persiste-os no PostgreSQL e, em seguida, emite um evento via WebSocket (Supabase Realtime) para o *dashboard*. O *dashboard* atualiza o gráfico de temperatura em tempo real.

Figura 4 - Trecho de tópicos MQTT usados para telemetria:

```
telemetry/{device_id}/temperature
telemetry/{device_id}/humidity
telemetry/{device_id}/presence
```

Fonte: Elaborado pelo autor (2026).

4.2 Fluxo de Controle Manual

O controle manual (acionado pelo gestor no *dashboard*) deve ser executado em até 2 segundos (RNF08). O comando humano sempre sobrescreve decisões automáticas (RN04).

Ar-condicionado: O *dashboard* envia uma requisição POST para `/api/v1/devices//control`.

O *backend* valida o *token* JWT, verifica as permissões do usuário e publica o comando no tópico MQTT `command/.../ac_01/setpoint`.

O ar-condicionado (atuador) recebe o comando, executa a ação e publica um *feedback* de confirmação no tópico `status/.../ac_01/feedback`.

O *backend* recebe essa confirmação, registra o comando na tabela `commands` com *status* "executed" e notifica o *dashboard*.

Figura 5 - Exemplo de comando via API e MQTT:

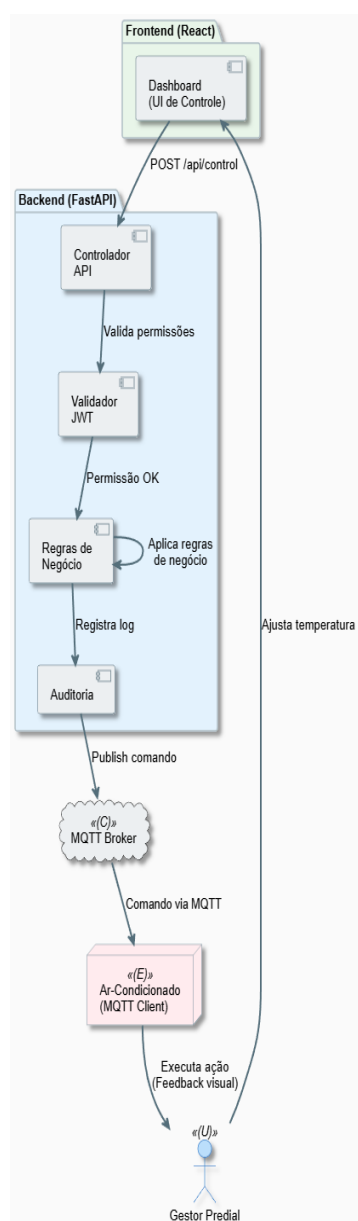
```
POST /api/v1/devices/{device_id}/control

command/{device_id}/setpoint
command/{device_id}/on
command/{device_id}/off

status/{device_id}/feedback
```

Fonte: Elaborado pelo autor (2026).

4.2.1 Figura 6 – Diagrama de Sequência – Coleta de manual



Fonte: Elaborado pelo autor (2026).

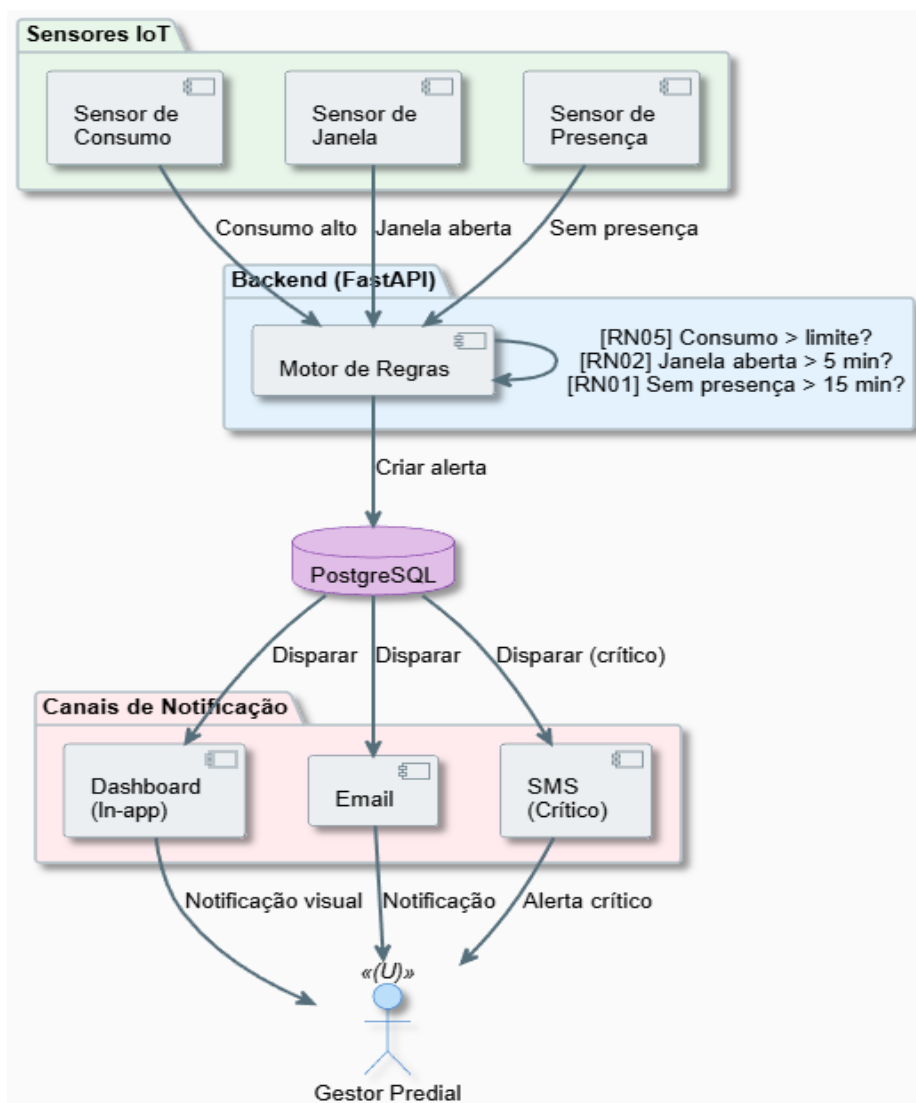
Descrição da figura: O diagrama mostra o gestor interagindo com o *dashboard* para ligar um dispositivo.

4.3 Fluxo de Alertas

O sistema monitora continuamente as leituras e, ao detectar condições anômalas (superação de limiares, anomalias do *Isolation Forest*), gera alertas e notifica os usuários.

4.3.1 Figura 7 – Diagrama de Sequência – Fluxo de Alertas

Descrição da figura: O *backend* recebe uma leitura de temperatura (32 °C) que excede o limiar configurado (30 °C). O módulo de regras de negócio identifica a violação e insere um alerta na tabela *alerts* com severidade "critical". Em seguida, o *backend* notifica o *dashboard* via WebSocket e, paralelamente, envia um e-mail ao gestor utilizando um serviço SMTP (ex.: SendGrid).



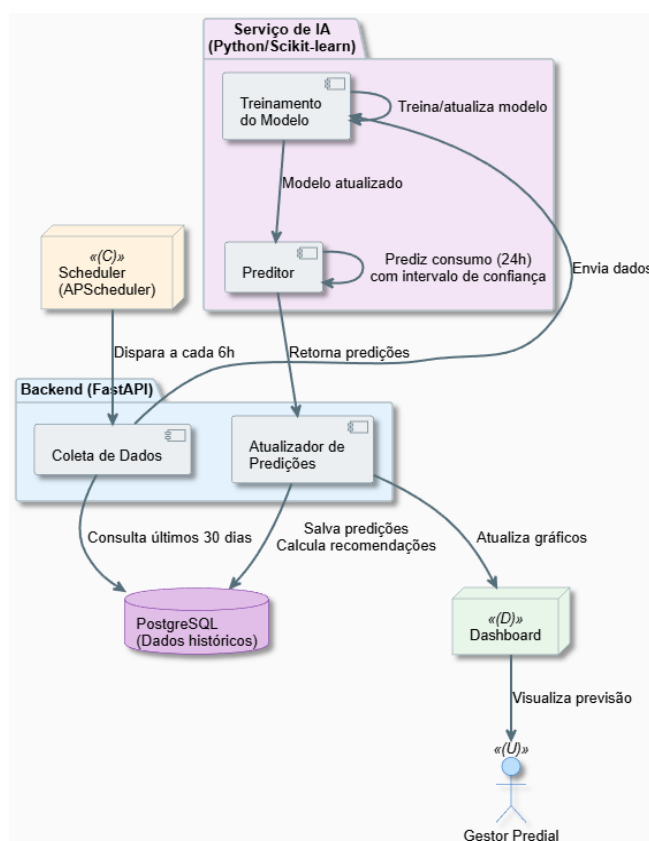
Fonte: Elaborado pelo autor (2026).

4.4 Fluxo de Predição com IA

O módulo de IA é executado diariamente às 02h00 (via *Scheduler*). Ele coleta os dados dos últimos 30 dias, retreina o modelo de regressão e gera previsões para as próximas 24 horas.

4.4.1 Figura 8 – Diagrama de Sequência – Predição com IA

Descrição da figura: Às 02h00, o *Scheduler* dispara o *job train_and_predict*. O *Serviço de IA* consulta o banco de dados pelos últimos 30 dias de consumo, temperatura externa e ocupação. Os dados são pré-processados (*feature engineering*) e divididos em treino (80%) e teste (20%). O modelo *LinearRegression* é treinado e avaliado (métricas MAE, RMSE, R^2). Em seguida, para cada hora das próximas 24 horas, o modelo gera uma previsão de consumo, que é armazenada na tabela *predictions*. Ao final, o serviço atualiza o *status* no *dashboard*.



Fonte: Elaborado pelo autor (2026).

5 MODELAGEM DE DADOS

5.1 Entidades Principais

O modelo de dados é composto pelas seguintes entidades, todas utilizando UUID como chave primária (exceto tabelas de alta volumetria, que utilizam BIGSERIAL):

5.1.1 Users (Usuários)

Tabela 4 – Definição da estrutura da entidade de usuários do sistema

Campo	Tipo	Descrição
id	UUID	Identificador único
email	VARCHAR(255)	Endereço de e-mail (unique)
password_hash	VARCHAR(255)	Hash da senha (bcrypt)
role	ENUM	'admin', 'operador', 'visualizador'
is_active	BOOLEAN	Indica se o usuário está ativo
last_login	TIMESTAMP	Data/hora do último acesso
created_at	TIMESTAMP	Data de criação

Fonte: Elaborado pelo autor

5.1.2 Rooms (Salas)

Tabela 5 – Definição da entidade de salas no modelo de dados

Campo	Tipo	Descrição
id	UUID	Identificador único
name	VARCHAR(100)	Nome da sala
building	VARCHAR(100)	Nome do edifício
floor	INTEGER	Andar
area_m2	NUMERIC(6,2)	Área em metros quadrados
max_occupancy	INTEGER	Ocupação máxima permitida

Fonte: Elaborado pelo autor

6 DEVICES (DISPOSITIVOS)

Tabela 6 – Definição da entidade de dispositivos e sensores do modelo de dados

Campo	Tipo	Descrição
id	UUID	Identificador único
room_id	UUID (FK)	Referência à sala
device_type	ENUM	'AC', 'temperature_sensor', 'humidity_sensor', 'presence_sensor'
model	VARCHAR(100)	Modelo do equipamento
serial_number	VARCHAR(50)	Número de série
mqtt_topic	VARCHAR(255)	Tópico MQTT base
status	ENUM	'online', 'offline', 'maintenance'
last_seen	TIMESTAMP	Último sinal de vida recebido

Fonte: Elaborado pelo autor

6.1 SensorData (Leituras dos Sensores)

Tabela 7 – Estrutura da entidade de dados de sensores e leituras

Campo	Tipo	Descrição
id	BIGSERIAL	Identificador sequencial
device_id	UUID (FK)	Referência ao dispositivo
temperature	NUMERIC(5,2)	Temperatura em °C
humidity	NUMERIC(5,2)	Umidade relativa em %
presence	BOOLEAN	Indicador de presença (se aplicável)
timestamp	TIMESTAMPTZ	Data/hora da leitura (com timezone)
is_anomaly	BOOLEAN	Indica se foi classificada como anômala

Fonte: Elaborado pelo autor

Índice recomendado: `CREATE INDEX idx_sensor_data_device_ts ON sensor_data(device_id, timestamp DESC);`

6.1.1 Commands (Comandos)

Tabela 8 – Estrutura da tabela de comandos de dispositivos e sensores

Campo	Tipo	Descrição
id	UUID	Identificador único
device_id	UUID (FK)	Referência ao dispositivo
user_id	UUID (FK)	Usuário que emitiu (NULL se automático)
command_type	ENUM	'on', 'off', 'setpoint'
value	NUMERIC(5,2)	Valor do set-point
issued_at	TIMESTAMPTZ	Data/hora de emissão
executed_at	TIMESTAMPTZ	Data/hora de execução confirmada
status	ENUM	'pending', 'executed', 'failed', 'timeout'

Fonte: Elaborado pelo autor

6.1.1.1 Alerts (Alertas)

Tabela 9 – Estrutura de campos da tabela de alertas de sensores

Campo	Tipo	Descrição
id	UUID	Identificador único
device_id	UUID (FK)	Referência ao dispositivo
alert_type	VARCHAR(50)	'high_temperature', 'low_humidity', 'device_offline', 'anomaly', 'high_consumption'
severity	ENUM	'info', 'warning', 'critical'
message	TEXT	Descrição legível do alerta
created_at	TIMESTAMPTZ	Data de criação
acknowledged_at	TIMESTAMPTZ	Data de reconhecimento (NULL se pendente)
resolved_at	TIMESTAMPTZ	Data de resolução (NULL se ativo)

Fonte: Elaborado pelo autor

Predictions (Predições)

Tabela 10 – Estrutura da tabela de dados de sensores e predições de consumo

Campo	Tipo	Descrição
id	UUID	Identificador único
device_id	UUID (FK)	Referência ao dispositivo (AC)
predicted_cons	NUMERIC(8,3)	Consumo predito em kWh
confidence	NUMERIC(5,2)	Índice de confiança (0 a 1)
timestamp	TIMESTAMPTZ	Data/hora da predição
actual_cons	NUMERIC(8,3)	Consumo real posterior (para retroalimentação)

Fonte: Elaborado pelo autor

6.1.1.2 AuditLog (Log de Auditoria)

Tabela 11 – Estrutura e descrição dos campos da tabela de auditoria de eventos

Campo	Tipo	Descrição
id	BIGSERIAL	Identificador sequencial
user_id	UUID (FK)	Referência ao usuário (NULL se sistema)
action	VARCHAR(100)	'LOGIN', 'COMMAND', 'SETPOINT_CHANGE', 'ALERT_ACK', 'USER_CREATED', etc.
resource	VARCHAR(100)	Recurso afetado ('device', 'user', 'room')
resource_id	UUID	Identificador do recurso
timestamp	TIMESTAMPTZ	Data/hora exata
metadata	JSONB	Payload completo em JSON

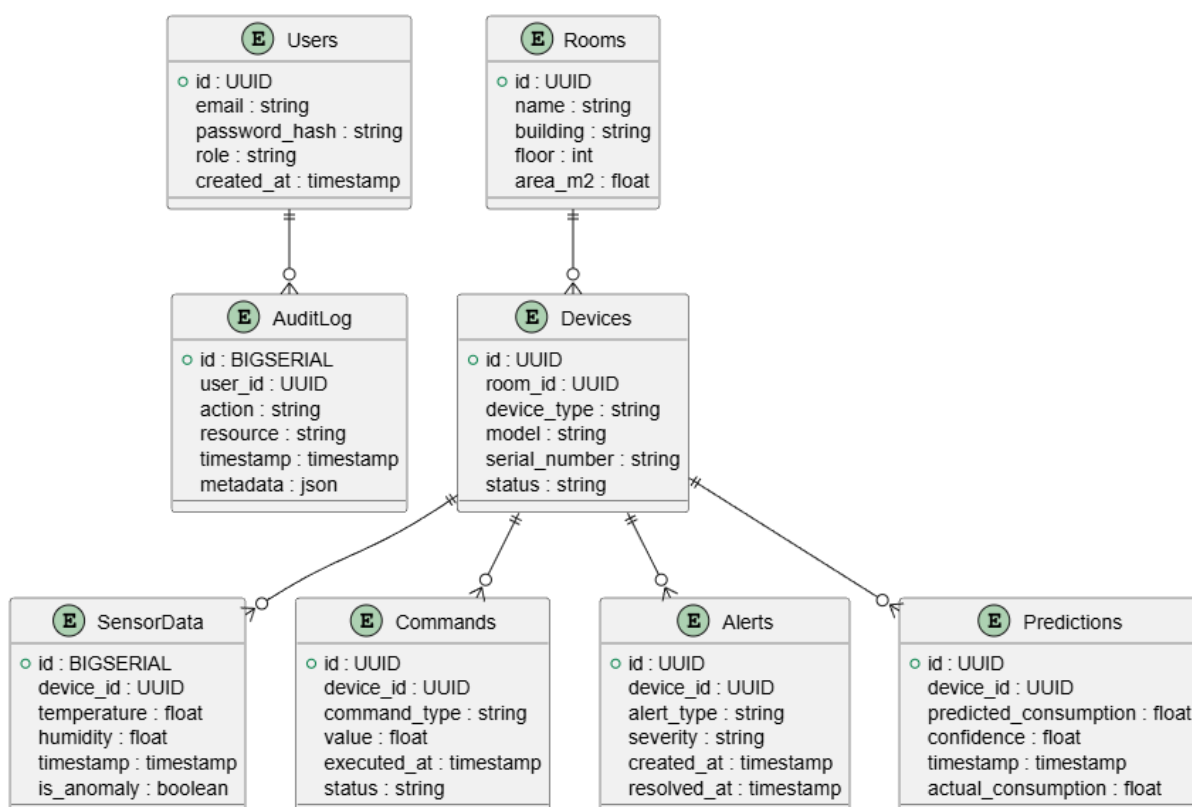
Fonte: Elaborado pelo autor

6.2 Relacionamentos

O Diagrama Entidade-Relacionamento (DER) a seguir ilustra as conexões entre as entidades.

6.2.1 Figura 9 – Diagrama Entidade-Relacionamento (DER)

Descrição da figura: O diagrama mostra oito tabelas interconectadas. A tabela rooms possui um relacionamento 1:N com devices. A tabela devices está no centro, com relacionamentos 1:N para sensor_data, commands, alerts e predictions. A tabela users está ligada 1:N a audit_log e também a commands (para registrar o emissor). Linhas com "1" e "∞" indicam as cardinalidades.



6.2.1.1 Quadro 3 – Relacionamentos entre Entidades

Tabela 12 – Relacionamentos e cardinalidades entre as entidades do sistema

Relacionamento	Cardinalidade	Descrição
Users → AuditLog	1:N	Um usuário realizou várias ações registradas
Users → Commands	1:N	Um usuário emitiu vários comandos
Rooms → Devices	1:N	Uma sala possui vários dispositivos
Devices → SensorData	1:N	Um dispositivo produz muitas leituras
Devices → Commands	1:N	Um dispositivo recebe muitos comandos
Devices → Alerts	1:N	Um dispositivo pode gerar múltiplos alertas
Devices → Predictions	1:N	Um dispositivo possui várias previsões

Fonte: Elaborado pelo autor (2026).

6.3 Ciclo de Vida e Retenção de Dados (Data Lifecycle)

Com uma latência de atualização de 5 segundos por sensor, a tabela `sensor_data` receberá aproximadamente 17.280 registros por dia para um único sensor ($12 \text{ leituras/min} \times 60 \text{ min} \times 24 \text{ h}$). Para um edifício com 48 sensores, o volume diário será de cerca de 829.440 registros, totalizando aproximadamente **25 milhões de linhas por mês**.

Para manter a performance e viabilidade de custos no Supabase, a seguinte política de *downsampling* (agregação progressiva) será implementada via rotinas do *Scheduler*:

- **Dados Quentes (0 a 7 dias):** Leituras brutas mantidas com granularidade total (5 em 5 segundos) na tabela `sensor_data`, permitindo análise em tempo real e *troubleshooting* de anomalias;
- **Dados Mornos (8 a 30 dias):** Agregação em médias de 15 minutos. Uma nova tabela `sensor_data_15min` armazena a média, o mínimo, o máximo e o desvio padrão de cada janela. Os registros brutos com mais de 7 dias são excluídos;
- **Dados Frios (após 30 dias):** Agregação em médias diárias consolidadas na tabela `sensor_data_daily`. Os registros de 15 minutos com mais de 30 dias são expurgados, mantendo o banco de produção leve e otimizando o custo de armazenamento (*storage* do Supabase).

A política de retenção é implementada por uma tarefa diária do *Scheduler* (03h00) que executa os comandos de agregação e expurgo dentro de uma transação.

6.4 Políticas de Segurança dos Dados

Em conformidade com a Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018), o sistema adota as seguintes práticas:

- **Minimização:** apenas dados estritamente necessários são coletados (temperatura, umidade, presença). Nenhuma informação pessoal identificável é capturada pelos sensores IoT;
- **Criptografia em repouso:** o PostgreSQL no Supabase utiliza criptografia AES-256 para dados em disco;
- **Criptografia em trânsito:** todas as conexões utilizam TLS 1.2+ (MQTT sobre TLS na porta 8883; HTTPS na porta 443);
- **Anonimização:** os dados utilizados para treinamento do modelo de IA são previamente anonimizados (remoção de `user_id` e `resource_id` sensíveis);
- **Direito ao esquecimento:** *soft delete* nas tabelas `users` e `audit_log`, permitindo exclusão definitiva mediante solicitação do titular.

7 ESPECIFICAÇÃO DA API RESTFUL

7.1 Autenticação e Autorização

A API utiliza autenticação baseada em **JWT (JSON Web Token)** com *refresh tokens*, em conformidade com o padrão OAuth 2.0 *Authorization Code Flow* (simplificado para SPA). Todos os *endpoints* (exceto */api/v1/auth/login* e */api/docs*) exigem o cabeçalho *Authorization: Bearer <access_token>*.

7.1.1 POST */api/v1/auth/login*

7.1.1.1 Erros Possíveis:

- 400 Bad Request: *payload* inválido (Pydantic);
- 401 Unauthorized: credenciais inválidas;
- 429 Too Many Requests: *rate limiting* excedido (10 tentativas/min por IP).

POST */api/v1/auth/refresh* – Renova o *access token* utilizando o *refresh token*.

POST */api/v1/auth/logout* – Invalida ambos os *tokens* (adiciona à *blacklist* no Redis).

8 ENDPOINTS PRINCIPAIS

8.1 Sensores

Tabela 13 – Endpoints da API para gerenciamento e consulta de sensores

Método	Rota	Descrição
GET	/api/v1/sensors	Lista todos os sensores (paginado)
GET	/api/v1/sensors/	Detalhes de um sensor específico
GET	/api/v1/sensors//data?period=24h	Dados históricos (1h, 24h, 7d, 30d)
GET	/api/v1/sensors//latest	Última leitura (temperature, humidity, timestamp, presence)

Fonte: Elaborado pelo autor

8.2 Dispositivos (AC)

Tabela 14 – Endpoints da API para gerenciamento e controle de dispositivos

Método	Rota	Descrição
GET	/api/v1/devices	Lista todos os ACs (filtro por status)
POST	/api/v1/devices//control	Controla um dispositivo
GET	/api/v1/devices//status	Status atual do AC

Fonte: Elaborado pelo autor

Requisição POST /api/v1/devices//control :

json

8.3 Alertas

Tabela 15 – Endpoints da API para gerenciamento de alertas

Método	Rota	Descrição
GET	/api/v1/alerts?status=active	Lista alertas ativos (filtros: severity, device_id)
POST	/api/v1/alerts//acknowledge	Reconhece um alerta
GET	/api/v1/alerts/history?days=30	Histórico dos últimos N dias

Fonte: Elaborado pelo autor

8.4 Consumo e Predição

Tabela 16 – Endpoints da API para consumo e predições de energia

Método	Rota	Descrição
GET	/api/v1/consumption?period=month	Consumo histórico (kWh, custo em R\$)
GET	/api/v1/predictions/24h	Predição das próximas 24 horas
GET	/api/v1/reports/consumption	Relatório completo exportável (CSV/PDF)

Fonte: Elaborado pelo autor

9 DOCUMENTAÇÃO SWAGGER

Todos os *endpoints* possuem documentação automática e interativa gerada pelo FastAPI, disponível nas seguintes rotas:

- GET /api/docs: Interface **Swagger UI** interativa (*try-out* disponível);
- GET /api/redoc: Documentação **ReDoc** (formato mais legível para leitura).

Cada *endpoint* documenta: descrição textual, parâmetros de entrada (*query*, *path*, *body*), códigos de resposta (200, 201, 400, 401, 403, 404, 500) e exemplos de requisição e resposta. A especificação segue o padrão **OpenAPI 3.0.3**.

10 COMPONENTES TÉCNICOS

10.1 Backend (FastAPI)

O *backend* é o núcleo orquestrador do sistema, responsável pela integração de todos os fluxos. Sua estrutura de diretórios segue os princípios de *Clean Architecture*:

Figura 10 – Árvore de diretórios do backend com módulos e arquivos FastAPI

```

backend/
├── app/
│   ├── api/
│   │   ├── routes/
│   │   │   ├── auth.py # Autenticação (login, refresh, logout)
│   │   │   ├── devices.py # CRUD e controle de dispositivos
│   │   │   ├── sensors.py # Consultas de sensores
│   │   │   ├── alerts.py # Gestão de alertas
│   │   │   ├── predictions.py # Predições e relatórios
│   │   │   └── users.py # Gestão de usuários (admin)
│   │   ├── dependencies.py # Dependências (get_db, get_current_user)
│   │   └── exceptions.py # Handlers de exceção globais
│   ├── core/
│   │   ├── config.py # Configurações (env vars, secrets)
│   │   ├── security.py # JWT, hashing, OAuth
│   │   └── constants.py # Enums, valores padrão
│   ├── models/ # Modelos SQLAlchemy ORM
│   ├── schemas/ # Schemas Pydantic (request/response)
│   ├── services/ # Lógica de negócio (regras RN01-RN10)
│   ├── db/
│   │   ├── session.py # Sessão do SQLAlchemy
│   │   └── migrations/ # Alembic migrations
│   ├── mqtt/
│   │   ├── client.py # Cliente MQTT (paho-mqtt)
│   │   └── topics.py # Constantes de tópicos
│   └── ai/
│       └── predictor.py # ConsumptionPredictor

```

Fonte: Próprio autor.

10.1.1 Principais Dependências (Requirements.txt)

fastapi0.104.1 uvicorn[standard]0.24.0 sqlalchemy2.0.23 psycpg2-binary2.9.9
 alembic1.13.0 pydantic[email]2.5.0 python-jose[cryptography]3.3.0
 passlib[bcrypt]1.7.4 paho-mqtt1.6.1 scikit-learn1.3.2 pandas2.1.3 numpy1.26.2
 schedule1.2.0 redis5.0.1 httpx0.25.2

10.2 Frontend (React)

python-multipart0.0.6

O *frontend* é uma *Single Page Application* (SPA) desenvolvida com React 18 e gerenciamento de estado via Zustand.

10.2.1 Estrutura de Componentes Principais

- **Dashboard:** visualização em tempo real com gráficos de linha (Recharts) para temperatura e consumo;
- **ControlPanel:** painel de controle com *cards* por sala, contendo controles *on/off* e *slider* de *set-point*;
- **AlertsPanel:** lista de alertas ativos com *badges* coloridos por severidade e botão de *acknowledge*;
- **ReportsPanel:** gerador de relatórios com filtros por período e exportação;
- **LoginPage:** formulário de autenticação com validação;
- **Layout:** *sidebar* de navegação, *header* com indicador de conexão e perfil do usuário.

10.2.2 Principais Bibliotecas (Package.json)

10.2.2.1

react: 18.2.0

react-router-dom: 6.20.0

axios: 1.6.2

recharts: 2.10.3 date-fns: 2.30.0 zustand: 4.4.6 tailwindcss: 3.3.6

@headlessui/react: 1.7.17

react-hot-toast: 2.4.1

11 MQTT – PADRÃO DE NOMENCLATURA DE TÓPICOS

Para garantir escalabilidade e facilitar o roteamento de mensagens, a arquitetura de tópicos adota uma taxonomia baseada na topologia física do edifício:

Formato geral:

11.1 Telemetria (Sensores)

Figura 10 - Trecho de definição de tópicos MQTT:

```
# Telemetria
telemetry/{device_id}/temperature
telemetry/{device_id}/humidity
telemetry/{device_id}/presence

# Comandos
command/{device_id}/setpoint
command/{device_id}/on
command/{device_id}/off

# Status
status/{device_id}/feedback
```

Fonte: Autoria própria (2026).

12 COMANDOS (ATUADORES):

command/building_a/floor_3/room_301/ac_01/setpoint

12.1 Status (Feedback):

status/building_a/floor_3/room_301/ac_01/feedback

Benefício arquitetural: Esta estrutura permite que o *backend* utilize *wildcards* MQTT para inscrições eficientes. Por exemplo, *telemetry/+/floor_3/#* recebe, com uma única subscrição, todos os dados de telemetria do terceiro andar.

12.2 Banco de Dados (PostgreSQL)

12.3 Configurações Recomendadas Para Ambiente de Produção (Supabase / VPS com 4 GB RAM):

Tabela 17 – Configurações de parâmetros do banco de dados PostgreSQL

Parâmetro	Valor
max_connections	200
shared_buffers	1 GB
effective_cache_size	3 GB
work_mem	16 MB
maintenance_work_mem	256 MB
wal_buffers	16 MB

Fonte: Elaborado pelo autor

12.3.1 Índices Adicionais Para Performance:

Figura 11 –Índices SQL para otimização de consultas

```
-- SensorData: consultas por dispositivo e janela temporal
CREATE INDEX idx_sensor_data_device_ts
ON sensor_data(device_id, timestamp DESC);

-- Commands: rastreamento de comandos pendentes
CREATE INDEX idx_commands_device_status
ON commands(device_id, status)
WHERE status = 'pending';

-- Alerts: listagem de alertas ativos por severidade
CREATE INDEX idx_alerts_active
ON alerts(device_id, created_at DESC)
WHERE resolved_at IS NULL;

-- Predictions: consulta das predições mais recentes
CREATE INDEX idx_predictions_device_ts
ON predictions(device_id, timestamp DESC);
```

Fonte: Autoria própria (2026).

Particionamento: A tabela `sensor\_data` será particionada por intervalo mensal na coluna `timestamp`, utilizando particionamento nativo do PostgreSQL 15+ (`PARTITION BY RANGE`).

13 INTELIGÊNCIA ARTIFICIAL

13.1 Modelo de Predição de Consumo

Objetivo: Prever o consumo energético (kWh) das próximas 24 horas para cada ar-condicionado, permitindo ao gestor antecipar demandas de pico e receber recomendações automáticas de economia.

Exemplo de treinamento e predição:

Figura 12 - Exemplo de treinamento e predição:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

Fonte: Autoria própria (2026).

13.1.1 Dados de Entrada (Features):

def predict_24h(self, current_data):

- e. Histórico de consumo dos últimos 30 dias (agregado em médias horárias);
- f. Temperatura externa (°C) obtida via API meteorológica (ex.: OpenWeatherMap);
- g. Previsão de ocupação do edifício (baseada em padrões históricos de presença);
- h. Hora do dia (0–23) e dia da semana (0–6);
- i. Indicador de final de semana (is_weekend: boolean);
- j. Indicador de feriado (is_holiday: boolean).

Algoritmo principal: Regressão Linear com scikit-learn, complementado opcionalmente por ARIMA para capturar sazonalidades não lineares (PEDREGOSA et al., 2011).

13.1.2 Métricas de Avaliação e Metas:

Tabela 18 – Métricas de avaliação e metas de desempenho do modelo de regressão

Métrica	Descrição	Meta
MAE	Erro Médio Absoluto	< 0.5 kWh
RMSE	Raiz do Erro Quadrático Médio	< 0.8 kWh

R ²	Coeficiente de Determinação	> 0.80
MAPE	Erro Percentual Absoluto Médio	< 15%

Fonte: Elaborado pelo autor

13.1.3 Política de Retreinamento:

- **Frequência:** diária, às 02h00 (horário de menor carga);
- **Janela de dados:** *rolling window* de 30 dias consecutivos;
- **Validação:** *hold-out* com 20% dos dados mais recentes como conjunto de teste;
- **Critério de rollback**:**** se o MAPE no teste for > 20%, o modelo anterior é mantido e um alerta é gerado para a equipe técnica.

13.2 Detecção de Anomalias

Para identificar padrões anormais nas leituras (ex.: sensor descalibrado, porta aberta gerando queda brusca de temperatura, falha intermitente), utiliza-se o algoritmo **Isolation Forest** (scikit-learn).

Diferentemente de abordagens com limiar fixo, o *Isolation*

Forest com `contamination='auto'` infere dinamicamente a proporção de anomalias com base na dispersão dos dados, reduzindo significativamente a taxa de falsos positivos.

Figura 13 – Trecho de código em Python para detecção de anomalias

```
from sklearn.ensemble import IsolationForest
from sklearn.ensemble import HistGradientBoostingRegressor
anomaly_detector = HistGradientBoostingRegressor()

anomalies = anomaly_detector.fit_predict(sensor_data)
```

Fonte: Autoria própria (2026).

14 TOPOLOGIA DE REDE IOT

14.1 Protocolos de Comunicação

O Quadro 4 lista os protocolos de rede utilizados em cada camada do modelo TCP/IP.

14.1.1 Quadro 4 – Protocolos de Comunicação Utilizados

Tabela 19 – Protocolos de rede utilizados em cada camada do modelo TCP/IP

Camada	Protocolo	Porta	Uso
Aplicação	MQTT	1883	Pub/Sub sensores (sem criptografia, apenas dev)
Aplicação	MQTT over TLS	8883	Pub/Sub com criptografia (produção)
Aplicação	HTTP	80	API REST (redireciona para HTTPS)
Aplicação	HTTPS	443	API REST segura (produção)
Aplicação	WebSocket (WSS)	443	Atualizações em tempo real
Transporte	TCP	–	Confiabilidade para MQTT/HTTPS
Transporte	UDP	–	Ping de status (leve)
Acesso	Modbus TCP	502	Controle direto de AC (quando disponível)
Acesso	WiFi 802.11b/g/n	–	Conectividade dos sensores ESP32
Acesso	Ethernet	–	Conectividade do gateway e backbone

Fonte: Elaborado pelo autor (2026).

14.2 Distribuição de Dispositivos

Exemplo de distribuição em edifício com 3 andares e 14 salas:

- **Andar 3** (5 salas: 301–305): 5 ACs, 5 sensores de temp., 5 sensores de umid., 2 sensores de presença;
- **Andar 2** (5 salas: 201–205): 5 ACs, 5 sensores de temp., 5 sensores de umid., 3 sensores de presença;
- **Andar 1** (4 salas: 101–104): 4 ACs, 4 sensores de temp., 4 sensores de umid., 1 sensor de presença.

Total do edifício: 14 ACs + 14 sensores de temperatura + 14 sensores de umidade + 6 sensores de presença = **48 dispositivos IoT**.

14.3 Camadas de Rede

A arquitetura de rede segue o modelo OSI adaptado:

- k. **Camada Física (1):** cabos Ethernet Cat6, antenas Wi-Fi integradas aos ESP32, modulação do sinal IR para acionamento dos ACs;
- l. **Camada de Enlace (2):** Wi-Fi 802.11/g/n para sensores, Ethernet para *gateway* e *backbone* do edifício;

- m. **Camada de Rede (3):** IPv4, DNS, roteamento entre VLANs (rede IoT segregada da rede corporativa);
- n. **Camada de Transporte (4):** TCP (confiável) para MQTT e HTTPS; UDP (leve) para pacotes de *keep-alive* e descoberta de dispositivos;
- o. **Camada de Aplicação (7):** MQTT, HTTP/HTTPS, WebSocket, Modbus TCP.

14.4 Recomendações de Hardware

14.4.1 Quadro 5 – Especificações de Hardware Recomendadas (100 dispositivos IoT)

Tabela 20 – Casos de teste principais baseados nos requisitos funcionais

Componente	Especificação Mínima
Ar-condicionados	Suporte a Modbus TCP ou controle IR (learning)

Fonte: Elaborado pelo autor (2026).

14.5 Especificação dos Módulos Sensores e Atuadores (Edge)

A camada física na ponta (*edge*) será composta por microcontroladores **ESP32**, escolhidos por possuírem conectividade Wi-Fi nativa, baixo consumo energético e capacidade de processamento local para *fallback offline*.

- **Sensoriamento:** leitura ambiental via sensores I^2C (ex.: BME280 para temperatura e umidade) e sensores PIR (HC-SR501) para presença. Cada ESP32 gerencia até 2 sensores de presença e 1 sensor ambiental;
- **Atuação:** para preservar a garantia dos fabricantes, os comandos serão emitidos de forma não invasiva, utilizando **diodos emissores de infravermelho (IR)** acoplados ao ESP32, que simulam os comandos do controle remoto original. O *firmware* do ESP32 é programado para "aprender" os códigos IR de cada modelo de AC durante a instalação (*setup por learning mode*).

14.6 Lógica de Resiliência (Offline Fallback)

Em caso de perda de conectividade com o *broker* MQTT (ex.: falha do Wi-Fi ou queda do enlace de internet), o nó ESP32 entrará no modo **Offline Fallback**, operando da seguinte forma:

- p. O dispositivo **congela a última configuração de temperatura** recebida antes da desconexão;
- q. O ESP32 passa a operar como um **termostato local "burro"**, utilizando suas próprias leituras do sensor de temperatura BME280 para ligar ou desligar o ar-condicionado via IR, mantendo a temperatura ambiente estável;

r. Ao reconectar-se à rede, o ESP32 publica um *buffer* com as leituras registradas durante o período *offline* (com *timestamps* locais) e retoma imediatamente o modo de operação normal.

Essa lógica garante a segurança dos ocupantes e evita superaquecimento ou resfriamento excessivo durante falhas de rede.

15 DEPLOYMENT E INFRAESTRUTURA

15.1 Plataformas de Hospedagem

15.1.1 Frontend (React) – Vercel

- *Deployment* automático via integração com repositório GitHub;
- *Build* otimizado com Vite;
- CDN global para distribuição de conteúdo estático (*Edge Network*);
- Certificado SSL/TLS provisionado automaticamente (Let's Encrypt);
- Plano *Hobby* (US\$ 20/mês) recomendado para produção (100 GB *bandwidth*).

15.1.2 Backend (FastAPI) – Render

- *Deploy* via contêiner Docker a partir do Dockerfile;
- Escala automática vertical (mais RAM/CPU) e horizontal (múltiplas instâncias) conforme demanda;
- Variáveis de ambiente gerenciadas com segurança (*sealed secrets*);
- Custo estimado: US\$ 12–20/mês para o plano *Standard* (512 MB RAM, 1 vCPU).

15.1.3 Banco de Dados (PostgreSQL) – Supabase

- PostgreSQL 15+ totalmente gerenciado, com *connection pooling* via pgBouncer;
- Autenticação JWT integrada ao ecossistema Supabase;
- Armazenamento: 8 GB no plano *Pro* (US\$ 25/mês), com *backups* diários automáticos;
- Suporte nativo a *WebSocket* (Realtime) para atualizações ao vivo no *dashboard*.

15.1.4 MQTT Broker – EMQX Cloud

- Suporte a MQTT 3.1.1 e 5.0;
- Capacidade: 1.000+ conexões simultâneas, 100.000+ mensagens/segundo;
- TLS 1.3 obrigatório, autenticação por certificado de cliente;
- Custo: a partir de US\$ 79/mês (plano *Starter*);
- Alternativa de menor custo: auto-hospedagem do Mosquitto 2 em VPS (DigitalOcean, Linode) a partir de US\$ 6/mês, exigindo gerenciamento próprio.

15.2 Docker Compose (Ambiente Local e Desenvolvimento)

O arquivo `docker-compose.yml` a seguir provisiona todo o ambiente local para desenvolvimento e testes:

```

version: '3.8'

services:
  backend:
    build: ./backend
    ports: ['8000:8000']
    environment:
      - DATABASE_URL=postgresql://smartbuilding:segura123@db:5432/smartbuilding
      - MQTT_BROKER=mqtt
      - MQTT_PORT=1883
      - JWT_SECRET=sua_chave_secreta_aqui
      - REDIS_URL=redis://redis:6379
    depends_on:
      db:
        condition: service_healthy
      mqtt:
        condition: service_started
    volumes:
      - ./backend/app:/app/app # Hot reload

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: smartbuilding
      POSTGRES_PASSWORD: segura123
      POSTGRES_DB: smartbuilding
    ports: ['5432:5432']
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U smartbuilding"]
      interval: 10s
      retries: 5

  mqtt:
    image: eclipse-mosquitto:2

```

```
ports:
  - '1883:1883'
  - '9001:9001' # WebSocket
volumes:
  - ./mosquitto.conf:/mosquitto/config/mosquitto.conf
  - mqtt_data:/mosquitto/data
redis:
  image: redis:7-alpine
  ports: ['6379:6379']
frontend:
  build: ./frontend
  ports: ['3000:3000']
  environment:
    - REACT_APP_API_URL=http://localhost:8000
  depends_on:
    - backend

volumes:
  postgres_data:
  mqtt_data:
```

16 PIPELINE DE CI/CD

O *pipeline* de integração e entrega contínua é executado via **GitHub Actions** e segue as etapas:

- s. **Code Push** para a *branch* main do repositório GitHub;
- t. **GitHub Actions** executa:
 - o *Linting*: ESLint + Prettier (frontend), Flake8 + Black (backend);
 - o *Testes unitários e de integração*: Jest (frontend) + Pytest (backend);
 - o *Build* da imagem Docker do *backend*;
- u. **Deploy do frontend** na Vercel (via *Vercel CLI* ou *GitHub Integration*);
- v. **Deploy do backend** no Render (via *Deploy Hook*);
- w. **Execução das migrações** do Alembic no banco de produção ();
- x. **Health check**: o *pipeline* aguarda 30 segundos e realiza uma requisição, validando se o *deploy* foi bem-sucedido;
- y. **Rollback automático**: se o *health check* falhar, o *pipeline* reverte para a última versão estável (*previous commit*).

17 ESTRATÉGIA DE TESTES

17.1 Testes Unitários

Os testes unitários cobrem as seguintes camadas:

- **Backend (Pytest):**
 - o *Models*: validação de integridade (*constraints, defaults, triggers*);
 - o *Schemas*: validação Pydantic para entrada/saída (ex.: e-mail inválido, valor de *setpoint* fora do intervalo);
 - o *Services*: cada regra de negócio (RN01 a RN10) é testada isoladamente com *mocks* para banco e MQTT;
 - o *Security*: geração, validação e expiração de tokens JWT;
 - o *AI*: funções de extração de *features*, normalização e predição.

17.1.1 Frontend (Jest + React Testing Library)

- *Componentes*: renderização condicional, eventos de clique, chamadas de API *mockadas*;
- *Estado*: transições do Zustand, atualização do *store* após ações.

Meta de cobertura: $\geq 80\%$ de *code coverage* (linhas).

17.2 Testes de Integração

- **Backend Banco de Dados**: Testes com banco de dados PostgreSQL real, provisionado via *testcontainers-python* (Docker efêmero);
- **Backend M**: Testes com *broker* Mosquitto local, simulando publicação de telemetria e recebimento de comandos (*pytest-mqtt*);
- **Frontend Backend**: Testes *end-to-end* com Playwright, simulando o fluxo completo: login → visualização do *dashboard* → emissão de comando → verificação do *feedback*.

17.3 Testes de Carga e Performance

Ferramenta: **Locust** (Python) ou **k6** (Grafana).

17.3.1 Cenários

- 48 sensores publicando telemetria a cada *5segundos* (carga nominal);
- 100 dispositivos simultâneos (pico de RNF02);
- 10 usuários simultâneos no *dashboard* realizando requisições GET e POST.

17.3.2 Critérios de Aceitação

- Latência p95 do *backend* < 500ms sob carga nominal;
- Zero *timeouts* nos comandos MQTT (QoS 1);
- *Dashboard* mantém taxa de atualização $\leq 5\text{segundos}$ (RNF01) com até 10 usuários.

17.4 Testes de Aceitação (UAT)

Os testes de aceitação são baseados nos requisitos funcionais e validados pelos *stakeholders*. O Quadro 7 lista os casos de teste principais.

17.4.1 Quadro 7 – Casos de Teste – Aceitação

Tabela 21 – Casos de teste principais baseados nos requisitos funcionais

ID	Cenário	Pré-condição	Resultado Esperado
UAT01	Gestor liga AC da Sala 301 pelo dashboard	AC está desligado	AC liga em $\leq 2s$, feedback "executed"
UAT02	Sala vazia por 15 minutos (RN01)	Sensor de presença = false	AC desliga automaticamente
UAT03	Janela aberta por 5 minutos (RN02)	Leitura de temperatura anômala	AC bloqueia e alerta é gerado
UAT04	Consumo excede limite configurado (RN05)	Limite = 10 kWh, real = 12 kWh	Alerta "critical" aparece no dashboard
UAT05	Predição das próximas 24h disponível	02h00 executou retreinamento	GET /predictions/24h retorna 24 valores
UAT06	Dashboard acessível em mobile (RNF09)	Navegador com viewport 375×812	Layout responsivo, controles adaptados

Fonte: Elaborado pelo autor (2026).

18 MONITORAMENTO E OBSERVABILIDADE

18.1 Métricas do Sistema

O monitoramento contínuo é essencial para garantir o SLA de 99% de disponibilidade (RNF05). O Quadro 8 lista as métricas coletadas.

18.1.1 Quadro 8 – Métricas de Monitoramento

Tabela 22 – Métricas de monitoramento e disponibilidade do sistema

Métrica	Ferramenta	Limiar de Alerta
Uptime do backend	UptimeRobot	< 99% no mês
Latência p95 da API REST	Grafana	> 1 segundo
Throughput MQTT (msg/s)	EMQX Dashboard	> 80% da capacidade
Uso de CPU (backend)	Prometheus	> 85% por 10 min
Conexões ativas no MQTT	EMQX API	< 48 (dispositivos) = alerta
Tamanho do banco (GB)	Supabase	> 70% do plano contratado
Erro 5xx na API	Sentry	Qualquer ocorrência

Fonte: Elaborado pelo autor (2026).

Stack de observabilidade: **Prometheus** (coleta) + **Grafana** (visualização) + **Sentry** (erros de aplicação) + **UptimeRobot** (disponibilidade externa).

18.2 Logging

- **Backend:** logs estruturados em JSON via structlog, incluindo: timestamp, level (INFO, WARNING, ERROR), service, trace_id (UUID único por requisição), message;
- **Centralização:** logs enviados para o **Supabase** (tabela application_logs) ou, alternativamente, para um *bucket* S3 com partição por data;
- **Retenção:** logs de aplicação retidos por 90 dias; logs de auditoria (audit_log) retidos por 5 anos (conformidade LGPD).

18.3 Alertas Operacionais

Alertas operacionais são distintos dos alertas de negócio (RN05). Eles são disparados para a equipe de DevOps/plataforma:

- *Backend down* (UptimeRobot): notificação via Slack e e-mail;
- *MQTT Broker inacessível:* o backend detecta *timeout* nas subscrições e publica um alerta na tabela alerts com severidade "critical";
- *Falha no retreinamento do modelo* (MAPE > 20%): e-mail para a equipe de dados;
- *Espaço em disco do banco < 15%:* notificação no Slack.

19 PLANO DE CONTINGÊNCIA E DISASTER RECOVERY

19.1 Análise de Riscos

O Quadro 9 mapeia os principais cenários de falha, probabilidade, impacto e resposta.

19.1.1 Quadro 9 – Plano de Contingência – Cenários e Respostas

Tabela 23 – Mapeamento de cenários de falha, probabilidade, impacto e resposta

Cenário	Probab.	Impacto	Resposta / Mitigação
Queda do broker MQTT (EMQX)	Baixa	Alto	ESP32 entra em Offline Fallback (seção 9.6). Backend tenta reconectar com backoff exponencial. Alerta crítico gerado.
Queda do backend (Render)	Baixa	Médio	Render reinicia automaticamente. Health check do CI/CD reverte deploy se necessário. Dashboard exibe banner "Reconectando".
Falha no banco de dados (Supabase)	Muito Baixa	Crítico	Restauração automática a partir do último backup (point-in-time recovery). Janela máxima de perda: 1 hora.
Perda de conectividade WiFi (andar)	Média	Médio	ESP32s afetados entram em Offline Fallback. Alerta "device_offline" gerado após 2 minutos sem sinal.
Falha no deploy (CI/CD)	Baixa	Médio	Rollback automático para a release anterior se health check falhar.
Ataque DDoS na API	Baixa	Alto	Vercel (frontend) e Render (backend) possuem proteção DDoS básica. Rate limiting configurado (100 req/min por IP).

Fonte: Elaborado pelo autor (2026).

19.2 Procedimentos de Recuperação

19.2.1 Queda do Broker MQTT

- EMQX Cloud possui SLA de 99.9%. Em caso de falha, a equipe aciona o suporte e, se necessário, migra para instância alternativa pré-configurada (*warm standby*);
- Dispositivos no *Offline Fallback* garantem operação local básica;
- Tempo de recuperação esperado (RTO¹): < 30 minutos.

19.2.2 Queda do Banco de Dados

¹ Recovery Time Objective —Objetivo de Tempo de Recuperação, tempo máximo aceitável para a recuperação de um serviço após uma falha ou desastre.

- O Supabase oferece *Point-in-Time Recovery* (PITR) no plano *Pro*. A última transação consistente é restaurada automaticamente;
- RTO: < 1 hora; RPO (*Recovery Point Objective*): 1 hora (último *backup* incremental).

19.2.3 Perda Total do Ambiente Cloud (Desastre)

- Todos os artefatos estão versionados (Git) e as imagens Docker estão no *GitHub Container Registry*;
- *Infrastructure as Code* (Docker Compose + scripts Terraform) permite recriar o ambiente em outro provedor (AWS, GCP) em < 4 horas.

19.2.4 Backups

Banco de dados: *backups* automáticos diários realizados pelo Supabase, retidos por 7 dias. *Backup* semanal completo armazenado em *bucket* S3 (*cold storage*), retido por 90 dias;

dias;

- **Configurações:** variáveis de ambiente e *secrets* armazenadas no *Vault* do Render, com exportação automática semanal;
- **Código-fonte:** repositório GitHub com histórico completo. Nenhum procedimento adicional necessário.

20 SEGURANÇA DA INFORMAÇÃO

20.1 Criptografia e Comunicação

- **Em trânsito:** TLS 1.2+ obrigatório para todas as comunicações externas (HTTPS na porta 443, MQTT sobre TLS na porta 8883). O certificado é provisionado e renovado automaticamente pelas plataformas (Vercel, Render, EMQX);
- **Em repouso:** criptografia AES-256 para dados em disco no PostgreSQL (Supabase) e nos volumes do Render;
- **Segredos:** JWT_SECRET, DATABASE_URL, MQTT_PASSWORD e chaves de API são armazenadas exclusivamente em variáveis de ambiente, nunca no código-fonte. Acesso restrito ao *pipeline* de CI/CD e ao *runtime* da aplicação.

20.2 Controle de Acesso

- **Autenticação:** JWT com *refresh token* e expiração de 1 hora. Senhas armazenadas com *hash bcrypt* (*cost factor* = 12);
- **Autorização:** modelo RBAC (*Role-Based Access Control*) implementado no *backend*:

Tabela 24 – Definição de permissões por papel no modelo RBAC

Role	Permissões
admin	Acesso total: CRUD de usuários, controle de dispositivos, relatórios
operador	Controle de dispositivos, visualização de alertas, reconhecimento
visualizador	Acesso somente leitura a dashboard e relatórios

Fonte: Elaborado pelo autor

Rate limiting: o *middleware* no FastAPI limita requisições a 100 por minuto por endereço IP, prevenindo abusos e ataques de força bruta.

20.3 Conformidade com a LGPD

O sistema processa dados de sensores ambientais (temperatura, umidade, presença), que, isoladamente, **não constituem dados pessoais** segundo a LGPD.

No entanto, os seguintes cuidados foram tomados para garantir *compliance*:

- **Dados de usuários (users):** e-mail e *hash* de senha são os únicos dados pessoais armazenados, com consentimento registrado no momento do cadastro;
- **Trilha de auditoria (audit_log):** todas as ações são registradas com identificação do usuário responsável, atendendo ao princípio da responsabilização;

- **Direito de acesso e exclusão:** *endpoints* administrativos permitem a consulta e exclusão definitiva dos dados de um usuário, mediante solicitação;
- **Termos de uso e política de privacidade:** disponíveis no *frontend* e apresentados no primeiro acesso ao sistema.

21 DECISÕES DE PROJETO

21.1 Por que MQTT?

21.1.1 Alternativas Avaliadas e Motivos de Descarte

- **HTTP REST:** latência elevada para aplicações IoT com *polling* frequente; *overhead* de cabeçalhos HTTP para mensagens pequenas; não suporta *push* nativo (requer *long polling* ou WebSocket separado);
- **WebSocket puro:** embora ofereça comunicação *full-duplex*, requer que o servidor mantenha conexão persistente com cada dispositivo, o que é inviável para 48+ dispositivos em termos de recursos;
- **CoAP:** suporte limitado em ferramentas de nuvem e ecossistema menos maduro que o MQTT; bibliotecas para Python e React são escassas.

Vantagens do MQTT: protocolo binário leve com *overhead* mínimo (2 bytes de cabeçalho); comunicação assíncrona *Pub/Sub* desacoplada; suporte nativo a Qualidade de Serviço (QoS 0, 1 e 2); excelente adequação para ambientes IoT com *bandwidth* restrito; comunidade e ecossistema maduros (MQTT FOUNDATION, 2026).

21.2 Por que FastAPI?

21.2.1 Alternativas:

- **Django:** *framework* completo, porém excessivamente pesado e opinativo para microsserviços; *Django REST Framework* adiciona complexidade desnecessária para APIs leves;
- **Flask:** ausência de suporte nativo a operações assíncronas (*async/await*), o que seria crítico para lidar com 48 conexões MQTT simultâneas;
- **Node.js (Express):** ecossistema JavaScript maduro, mas a integração com bibliotecas de IA/ML em Python (Scikit-learn, Pandas) exigiria *wrappers* ou processos separados, aumentando a complexidade.

Vantagens do FastAPI: suporte nativo a *async/await* com alto desempenho via Uvicorn; geração automática de documentação Swagger/ReDoc (OpenAPI 3.0); validação de dados integrada com Pydantic; *performance* comparável a Node.js e Go (TIANGOLO, 2026); integração direta com o ecossistema Python de *data science*.

21.3 Por que React?

21.3.1 Alternativas:

- **Vue.js:** ecossistema menor em comparação ao React, especialmente para componentes de *dashboard* complexos (gráficos, *tables*, *hooks* customizados);
- **Angular:** complexidade excessiva para o escopo do projeto; curva de aprendizado íngreme que impactaria a produtividade da equipe;
- **Svelte:** tecnologia emergente e promissora, porém com ecossistema de componentes de UI menos maduro.

Vantagens do React: maior comunidade e suporte global (META PLATFORMS, 2026); amplo ecossistema de componentes (Material-UI, Ant Design, Tailwind UI); gerenciamento de estado consolidado (Zustand para estados locais, React Query para *server state*); facilidade de *deploy* na Vercel com *preview deployments* por *branch*.

21.4 Por que Scikit-learn?

21.4.1 Alternativas:

- **TensorFlow / Keras:** *overhead* excessivo para problemas de regressão linear e séries temporais simples; aumentaria o tamanho da imagem Docker em > 1 GB;
- **PyTorch:** igualmente pesado para o escopo; a flexibilidade extra não se justifica para um problema de predição de consumo;
- **Implementação from scratch:** risco de introduzir *bugs* e ineficiências; reinventar a roda quando existem bibliotecas maduras e testadas pela comunidade.

Vantagens do Scikit-learn: biblioteca leve (~50 MB) e de alta performance para *machine learning* clássico; API intuitiva e extensivamente documentada; excelente suporte a algoritmos de regressão, classificação e detecção de anomalias; integração direta com NumPy e Pandas (PEDREGOSA et al., 2011). A utilização de `contamination='auto'` no *Isolation Forest* demonstra maturidade na escolha dos hiperparâmetros, evitando falsos positivos e adaptando-se dinamicamente à qualidade dos dados de cada edifício.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6022**: Informação e documentação – Artigo em publicação periódica técnica e/ou científica – Apresentação. Rio de Janeiro: ABNT, 2018a.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 6023**: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018b.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 10520**: Informação e documentação – Citações em documentos – Apresentação. Rio de Janeiro: ABNT, 2002.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 14724**: Informação e documentação – Trabalhos acadêmicos – Apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm . Acesso em: 28 abr. 2026.

BROWN, S. **The C4 Model for Visualising Software Architecture**. Disponível em: . Acesso em: 28 abr. 2026.

META PLATFORMS. **React – A JavaScript library for building user interfaces**. Disponível em: . Acesso em: 28 abr. 2026.

MQTT FOUNDATION. **MQTT: The Standard for IoT Messaging**. Disponível em: . Acesso em: 28 abr. 2026.

PEDREGOSA, F. *et al.* Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011. Disponível em: . Acesso em: 28 abr. 2026.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL: The World's Most Advanced Open Source Relational Database**. Disponível em: . Acesso em: 28 abr. 2026.

TIANGOLO, S. **FastAPI Documentation**. Disponível em: . Acesso em: 28 abr. 2026.

U.S. ENERGY INFORMATION ADMINISTRATION. **Commercial Buildings Energy Consumption Survey (CBECS)**. Washington, DC: EIA, 2023. Disponível em: . Acesso em: 28 abr. 2026.

Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 2026.